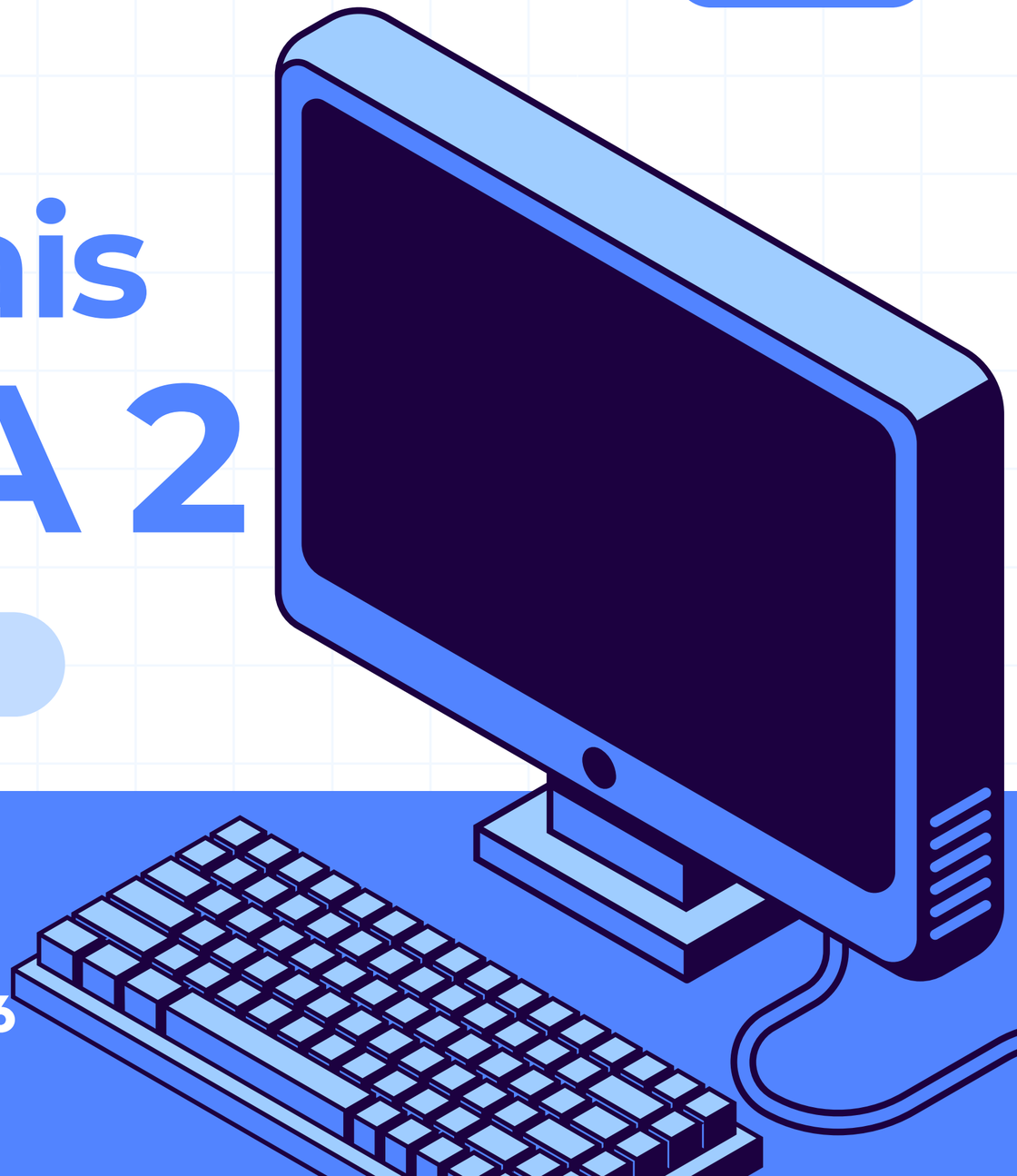
Sistemas Digitais
PROBLEMA 2

Coprocessador Gráfico



Cauan

Diego dos Santos

Heitor Abdalla

Date

14 de outubro de 2026

O Problema

Ponto de partida · Problema 1

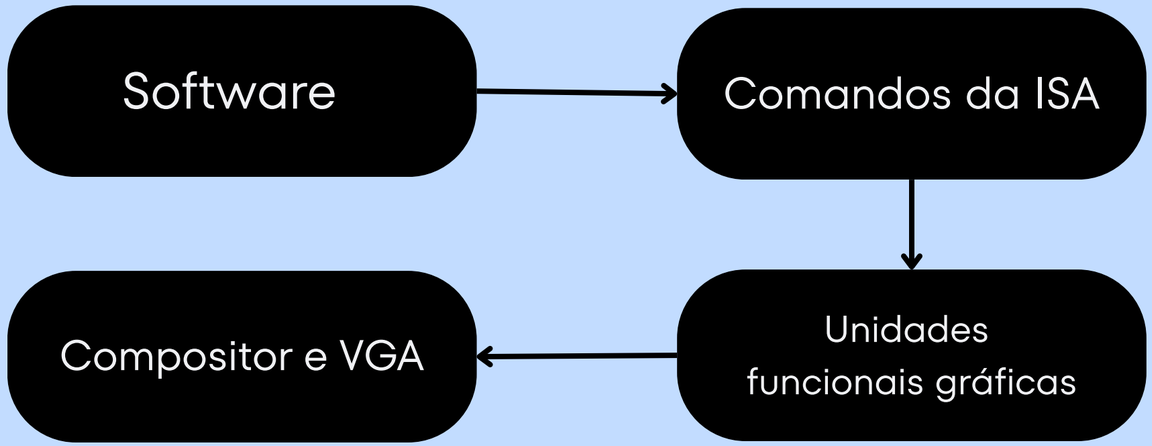
Motores independentes

Background, sprites e rasterização geram imagem, mas precisam ser controlados individualmente.

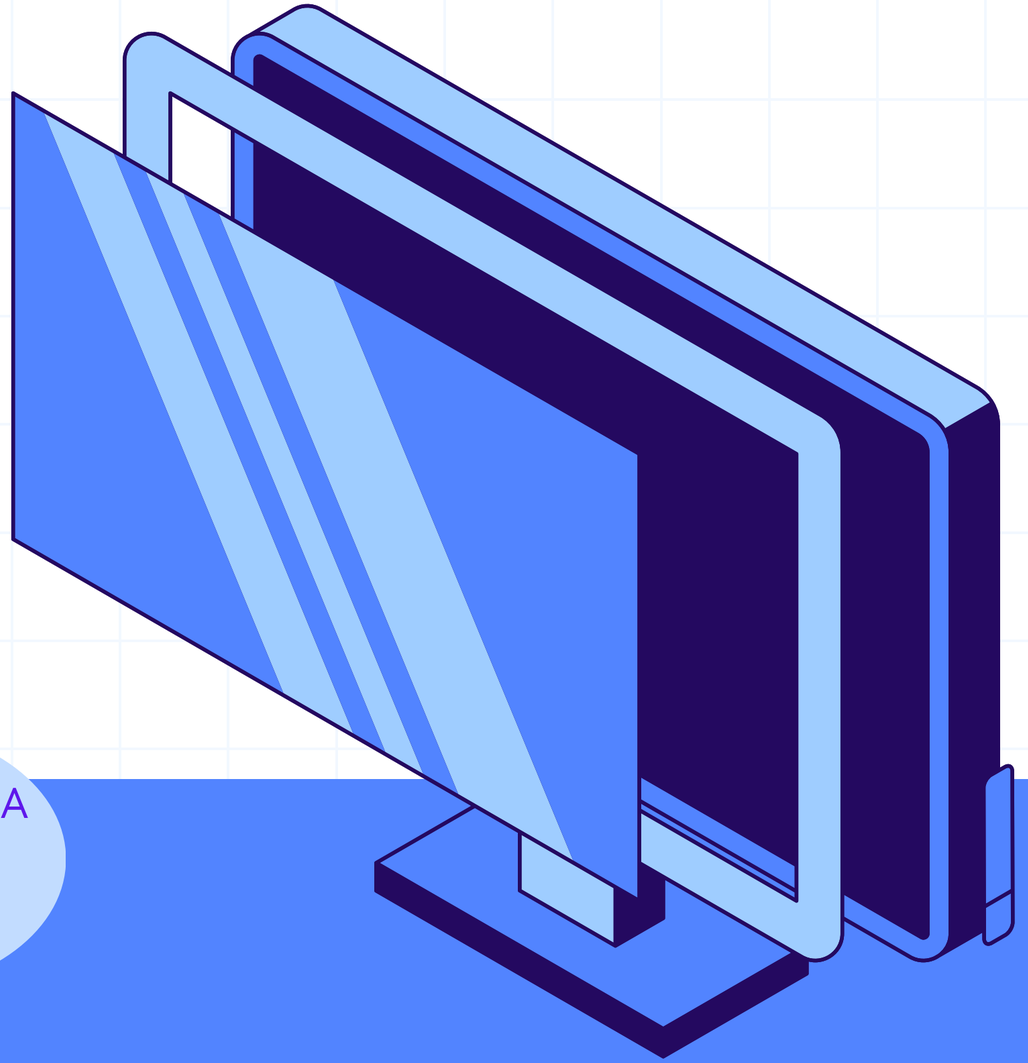
Objetivo · Problema 2

Coprocessador Programável

O software envia comandos; o hardware interpreta e aciona os recursos necessários.



Requisitos de Arquitetura



Núcleo programável (obrigatório)

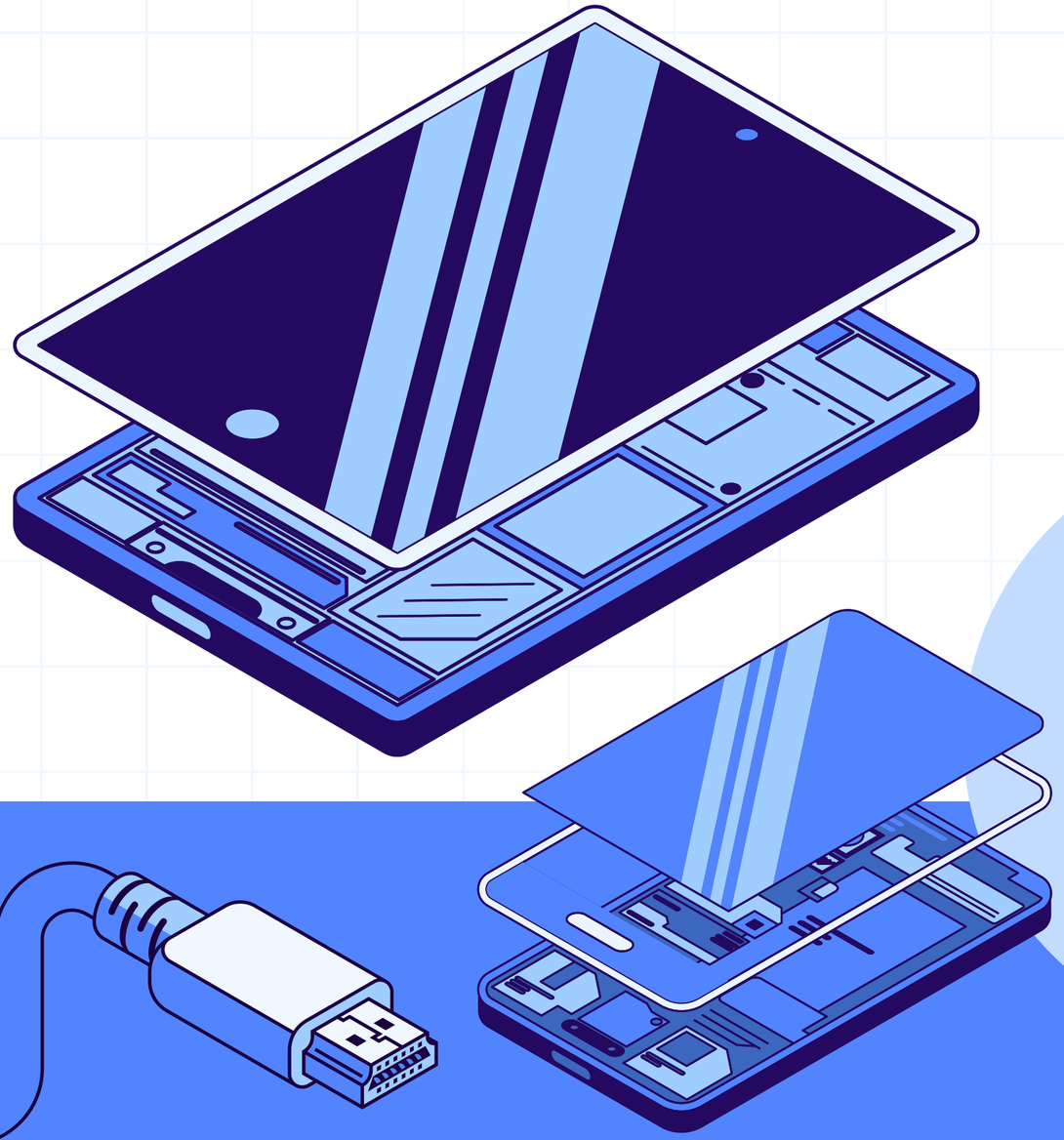
- ISA de 32 bits
- Unidade de busca de instruções
- Unidade de controle
- Datapath
- Banco de registradores
- Registrador de status
- Sincronização com a VGA

Unidades funcionais (mínimo)

- ULA
- Motor de background
- Motor de sprites
- Rasteizador de polígonos
- Unidade de controle de quadro
- Compositor
- Controlador de VGA

Toda operação é consequência da execução de uma instrução da ISA

Compositor, controle de quadro e VGA operam continuamente, independentes das instruções.



Programa, Verificação e Entrega

Programa gráfico

- Programa em Assembly da própria ISA
- Sintaxe definida pelo grupo
- Busca ativa: memória interna (.mif ou .hex)
- Busca passiva: fila de comandos via MMIO com o ARM

Verificação

- Testbench para cada módulo principal
- Testbench de integração do coprocessador
- Relatório de síntese: recursos, frequência máxima e timing

Entrega

- RTL completo e top-level da DE1-SoC
- Projeto Quartus compilável
- Repositório GitHub com README completo
- Demonstração em hardware na placa

Requisitos no repositório

Requisito	Onde está
ISA de 32 bits	rtl/include/isa.vh
Unidade de busca	rtl/core/fetch_unit/
Unidade de controle	rtl/core/control_unit/unidade_de_controle.v
Banco de registradores	rtl/core/registrars_bank/banco_registradores.v
ULA	rtl/core/alu.v
Registrador de status	FLAGS (Z, N): registrador 36 do espaço unificado
Datapath e integração	rtl/coprocessador_top.v
Unidades gráficas	rtl/motores_independentes
Compositor e VGA	compositor.v e rtl/driver/vga_driver.v
Sincronização com a VGA	HALT fecha o quadro; a troca de buffers ocorre no vblank

Arquitetura implementada

Coprocessador

O projeto implementa um coprocessador gráfico em FPGA baseado em uma ISA própria de 32 bits, cuja execução é controlada por uma unidade de busca de instruções. O programa é armazenado na memória de instruções e executado por meio do contador de programa (PC), da unidade de controle, do banco de registradores e da ULA.

Busca Ativa

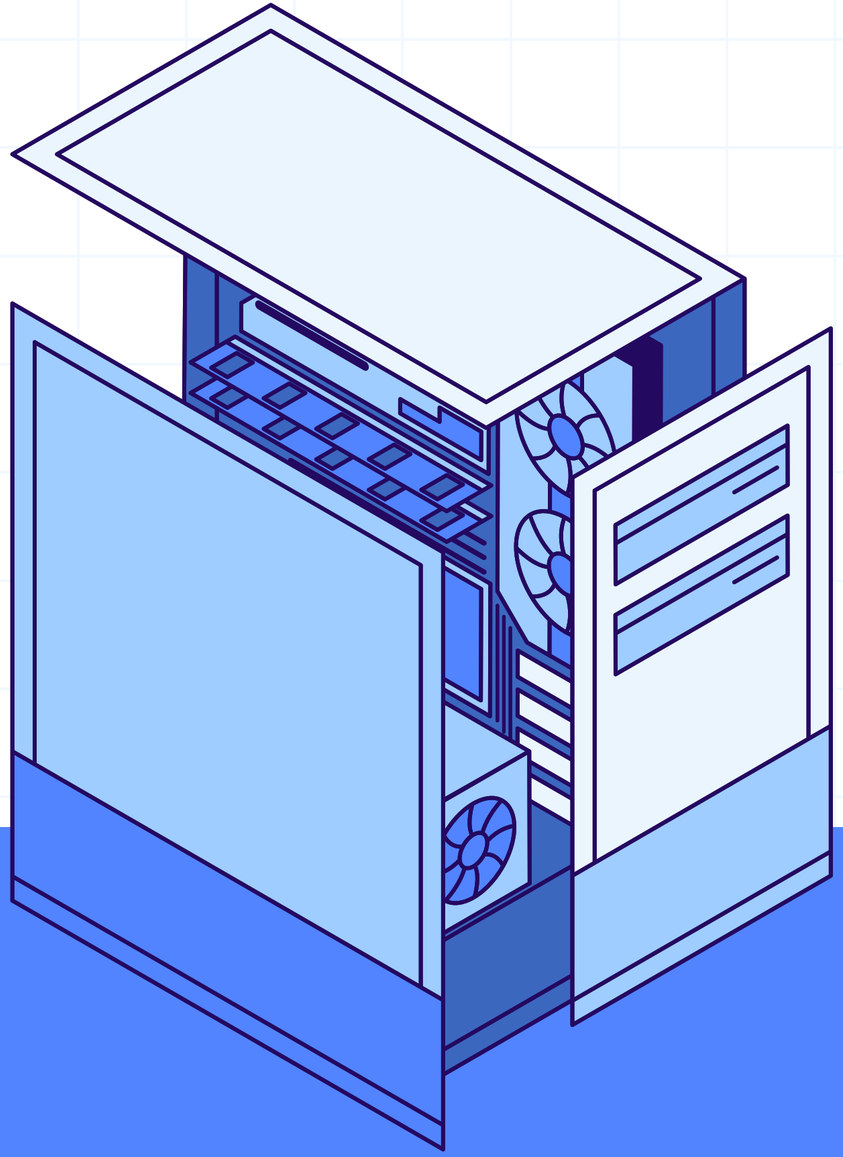
Consiste em executar continuamente um programa previamente carregado na memória, buscando, decodificando e executando suas instruções para controlar os recursos gráficos. O PC indica a posição da instrução a ser executada e é atualizado sequencialmente ou conforme o resultado de instruções de desvio.

Preservação de estado

O programa pode ser reiniciado no endereço inicial a cada intervalo de atualização vertical (VBlank), enquanto os registradores preservam o estado necessário entre quadros. Dessa forma, o programa pode utilizar variáveis de estado para produzir animações progressivas, calculando a posição e os atributos dos objetos em cada quadro.

Comunicação com motores gráficos

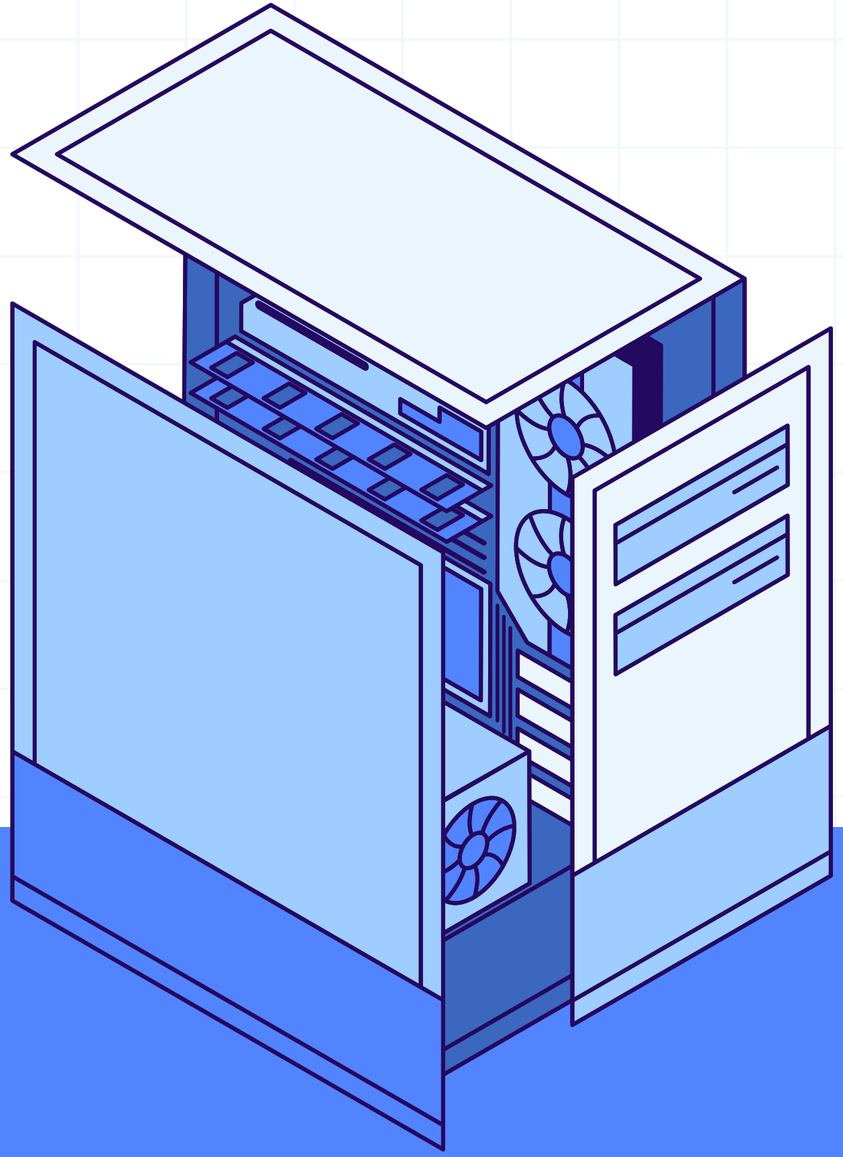
O núcleo de execução se comunica com os motores gráficos por meio de instruções específicas para configurar e desenhar o background, manipular sprites e rasterizar formas geométricas. Após a renderização, o sistema utiliza o framebuffer com double buffering: enquanto um banco é exibido pelo controlador VGA, o outro pode receber a nova imagem.



Datapath

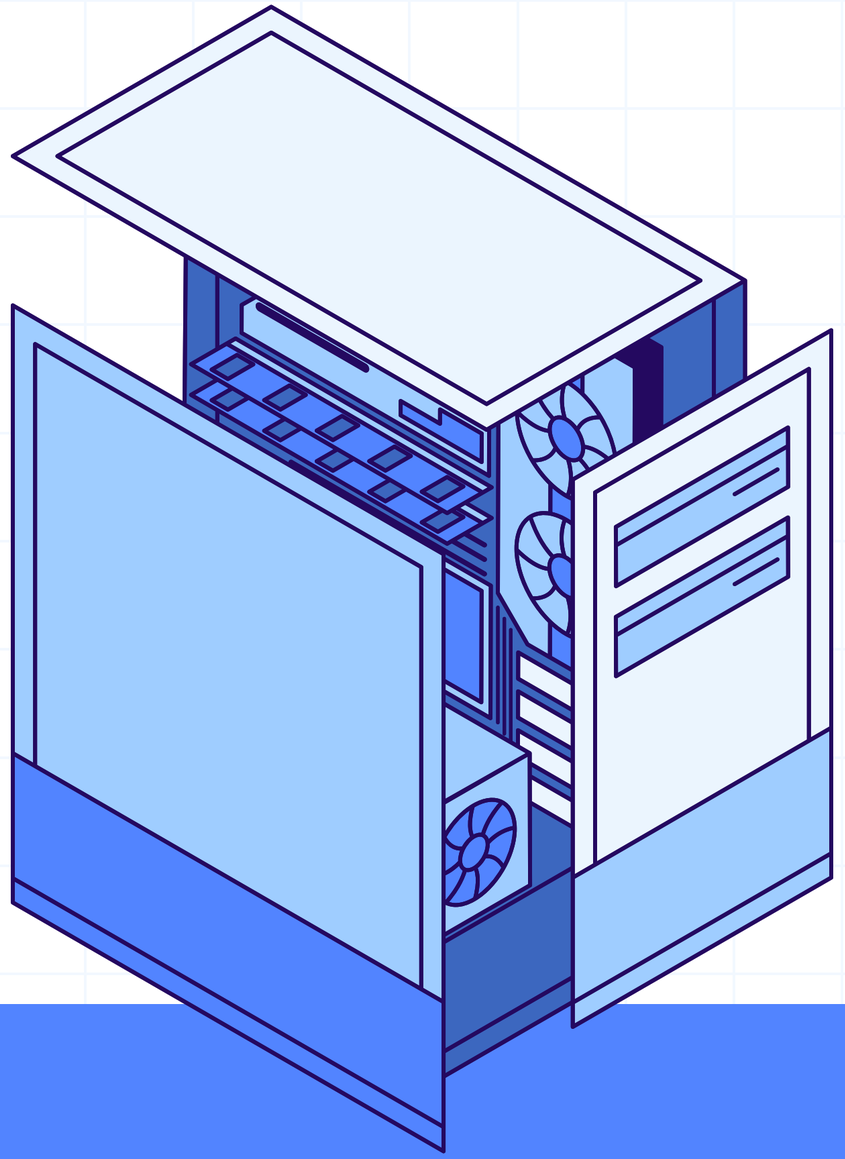
O datapath do coprocessador é composto pelos módulos responsáveis por armazenar, processar e transferir dados durante a execução das instruções e a geração da imagem.

O fluxo de dados começa na memória de instruções, que fornece palavras de 32 bits à unidade de busca. A unidade de controle decodifica o opcode e determina a operação a ser realizada, enquanto o banco de registradores disponibiliza os operandos necessários. Quando a instrução exige cálculos, a ULA processa os valores e atualiza as flags utilizadas nas decisões condicionais. Os resultados podem ser armazenados nos registradores ou utilizados para configurar os motores gráficos.



Visão geral do datapath

- Unidade de busca de instruções: fornece a instrução que será executada.
- Unidade de controle: decodifica o opcode e gera os sinais de controle.
- Banco de registradores: armazena operandos, parâmetros gráficos e resultados.
- ULA (ALU): executa operações aritméticas e lógicas.
- Memórias gráficas: armazenam paletas, mapas de tiles e padrões de pixels.
- Motores gráficos: desenham o cenário, os sprites e as primitivas geométricas.
- Multiplexador de motores: seleciona os dados do motor ativo.
- Camada de composição e framebuffer: organizam os pixels e armazenam a imagem para exibição.



Busca da instrução e fluxo de execução

- A unidade de busca (unidade_de_busca_de_instrucoes) mantém o controle do fluxo de instruções e fornece a instrução atual.
- A instrução possui 32 bits e contém campos como o opcode e os operandos ou parâmetros codificados.
- O sinal execute indica quando a instrução está na etapa de execução.
- A unidade de controle recebe instruction e execute e identifica a operação solicitada.
- Para operações gráficas longas, a busca recebe o sinal engine_done e aguarda a conclusão do motor selecionado antes de prosseguir.

Memória de instruções → Unidade de busca → instruction/execute → Unidade de controle

Bloco I: O Cérebro (Busca e Controle de Espera)

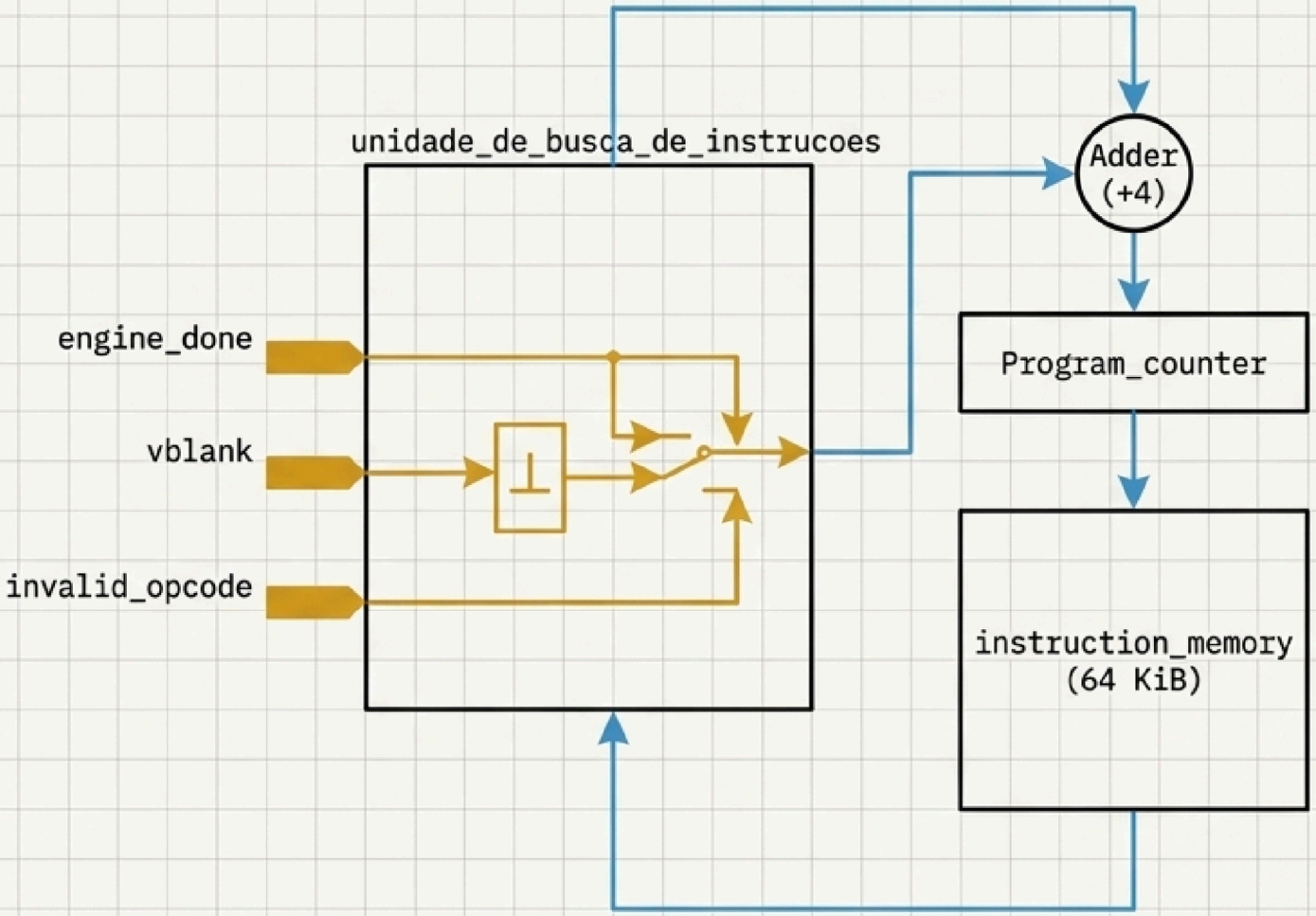
O Ciclo de Vida

A unidade_de_busca_de_instrucoes busca, emite o pulso execute e avança.

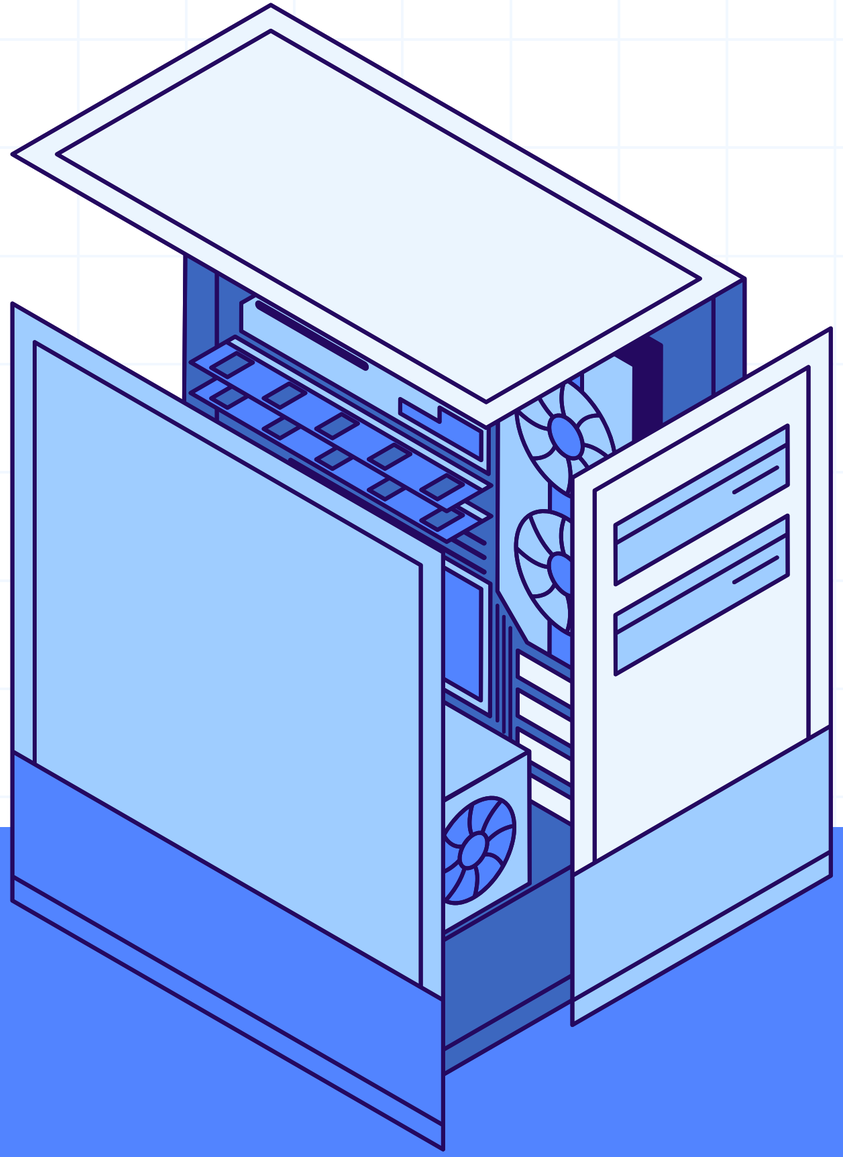
Sincronização Física

O avanço lógico não é cego. Ele interrompe o incremento do PC para esperar operações de hardware longas através de sinais bloqueantes:

- engine_done: Aguarda os motores terminarem de desenhar (DRAW_*).
- vblank: Aguarda o monitor terminar o quadro (WAIT_VBLANK).
- invalid_opcode: Sinaliza erros de formato.



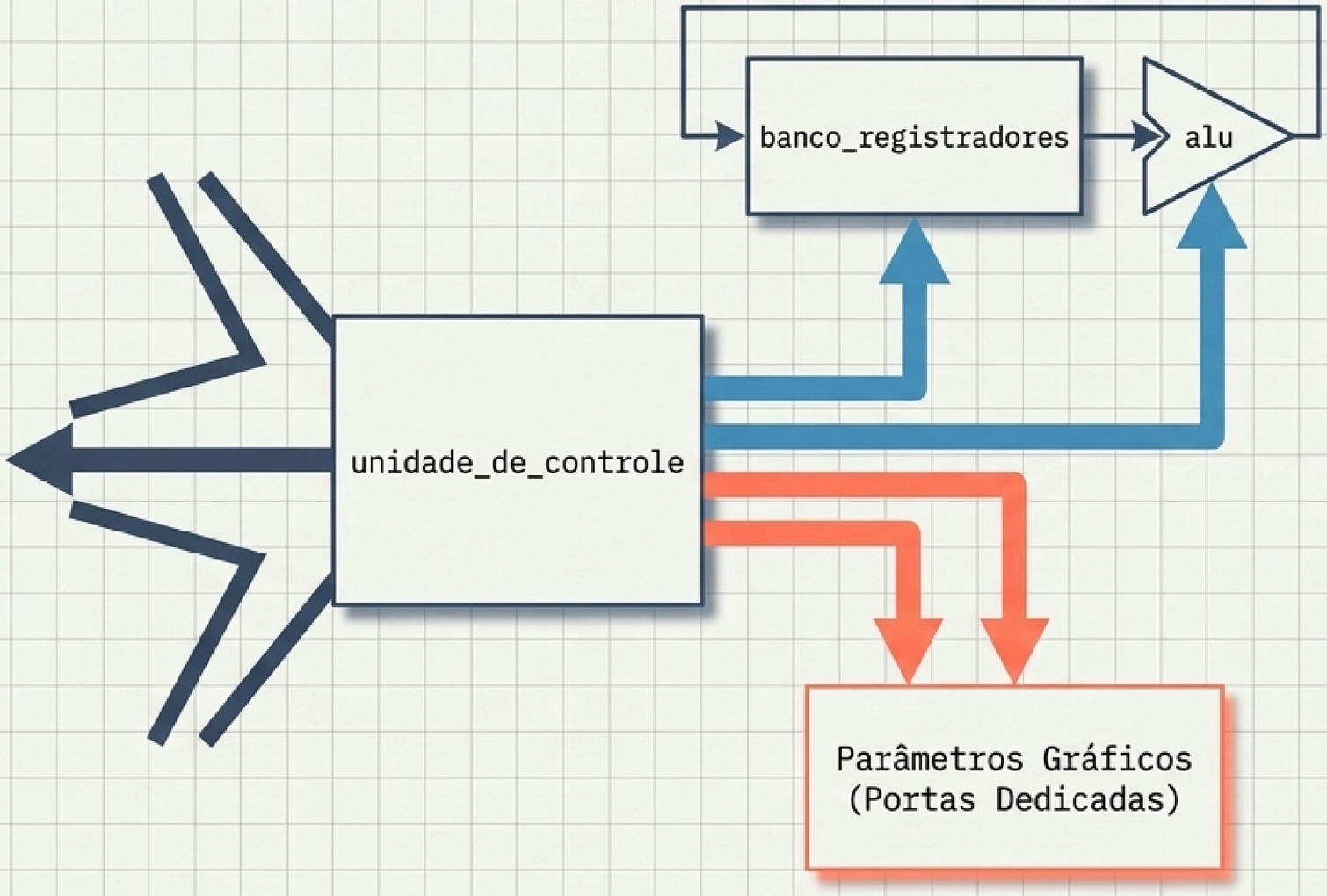
Gemini Notebo



Decodificação e geração dos sinais de controle

- A unidade de controle (unidade_de_controle) interpreta o campo opcode, localizado em instruction[31:27].
- A partir do opcode e do estado de execução, gera sinais para a ULA, banco de registradores, memórias gráficas e motores.
- Instruções aritméticas e lógicas ativam a ULA.
- Instruções de configuração escrevem dados em memórias ou registradores de controle.
- Instruções de desenho iniciam o motor correspondente.
- engine_sel identifica qual motor gráfico está ativo; engine_done sinaliza a conclusão da operação selecionada.

Bloco I: Caminho de Dados e Despacho



Caminho 1: Ciclo Aritmético

Banco de Registradores -> ULA (alu) -> Banco de Registradores.

Atualiza flags (Z, N, V, C) que são avaliadas pelo avalia_condicao para desvios no código.

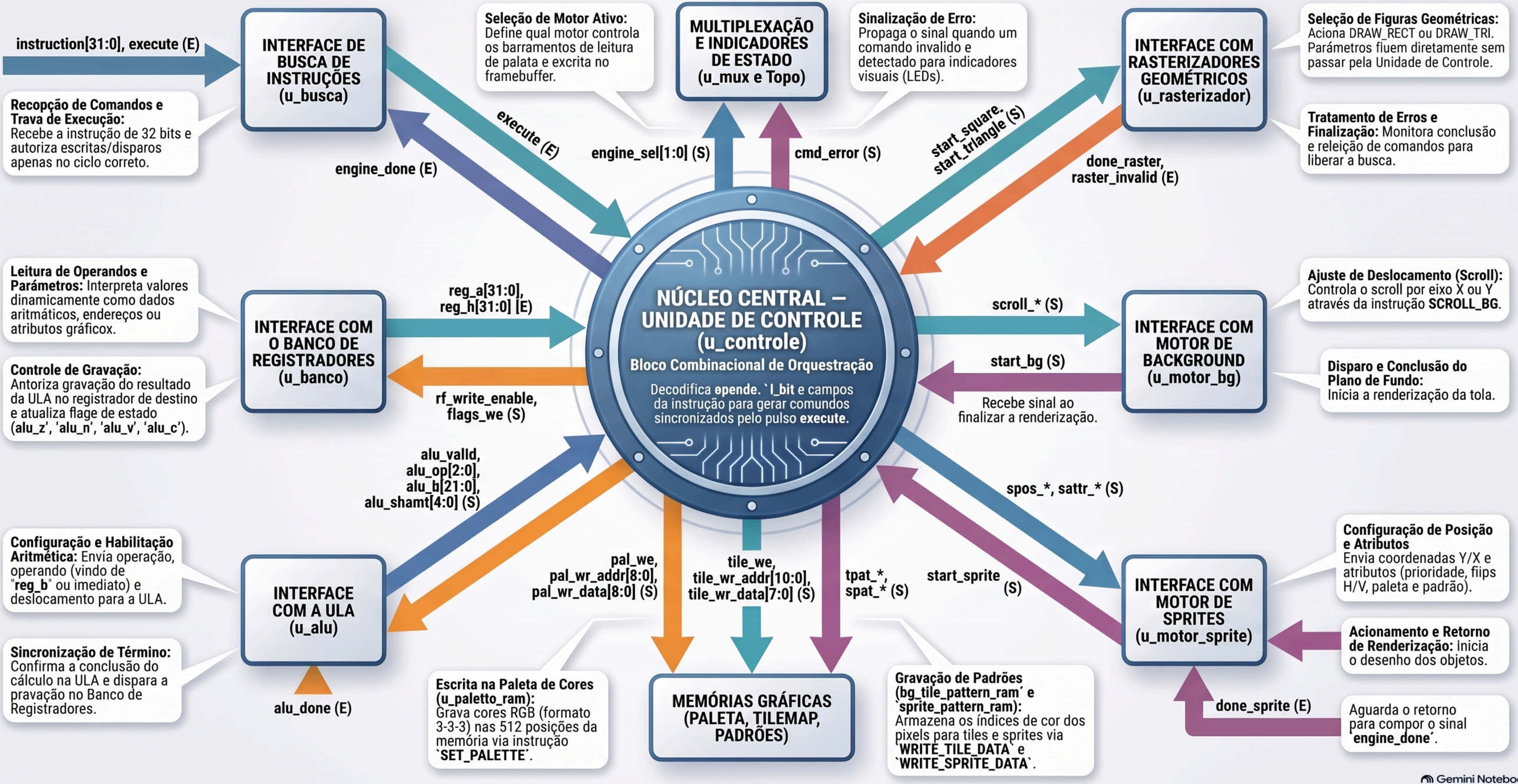
Caminho 2: Despacho Gráfico

A Unidade de Controle atua como despachante direto. O banco disponibiliza portas dedicadas para hardware gráfico que não custam ciclos extras do processador:

`v0x, v0y, v1x, v1y, v2x, v2y`	: Coordenadas de vértices
`color_index & palette_sel`	: Injetados no rasterizador

Gemini Notebo

Arquitetura da Unidade de Controle: Fluxo de Sinais e Conexões



Gemini Notebo

Banco de registradores e seleção dos operandos

- O banco de registradores (u_banco) fornece os operandos lidos pela ULA por meio de reg_a e reg_b.
- A instrução determina quais registradores serão acessados.
- Esses valores também podem representar endereços, cores, posições de sprites e outros parâmetros gráficos.
- A unidade de controle seleciona a origem do segundo operando da ULA:
- i_bit = 0: utiliza reg_b.
- i_bit = 1: utiliza o imediato de 12 bits, instruction[11:0], com extensão por zeros para 32 bits.
- O banco de registradores também recebe resultados da ULA e mantém os parâmetros utilizados pelo rasterizador.

Organização do Banco de Registradores

USO GERAL: 31 REGISTRADORES FLEXÍVEIS

R0–R30
32 bits

Operandos, temporários, coordenadas sem função fixa.

REGISTRADOR XZR: VALOR ZERO FIXO

R31

Retorna zero na leitura, descarta escritas. Atualiza flags sem salvar.

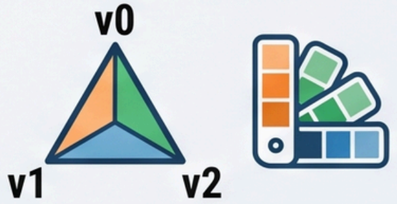
BANCO DE REGISTRADORES (u_banco): INTERFACE CENTRAL

Conecta ULA e Motor Gráfico

FLAGS E CONTROLE: ESTADO DO SISTEMA

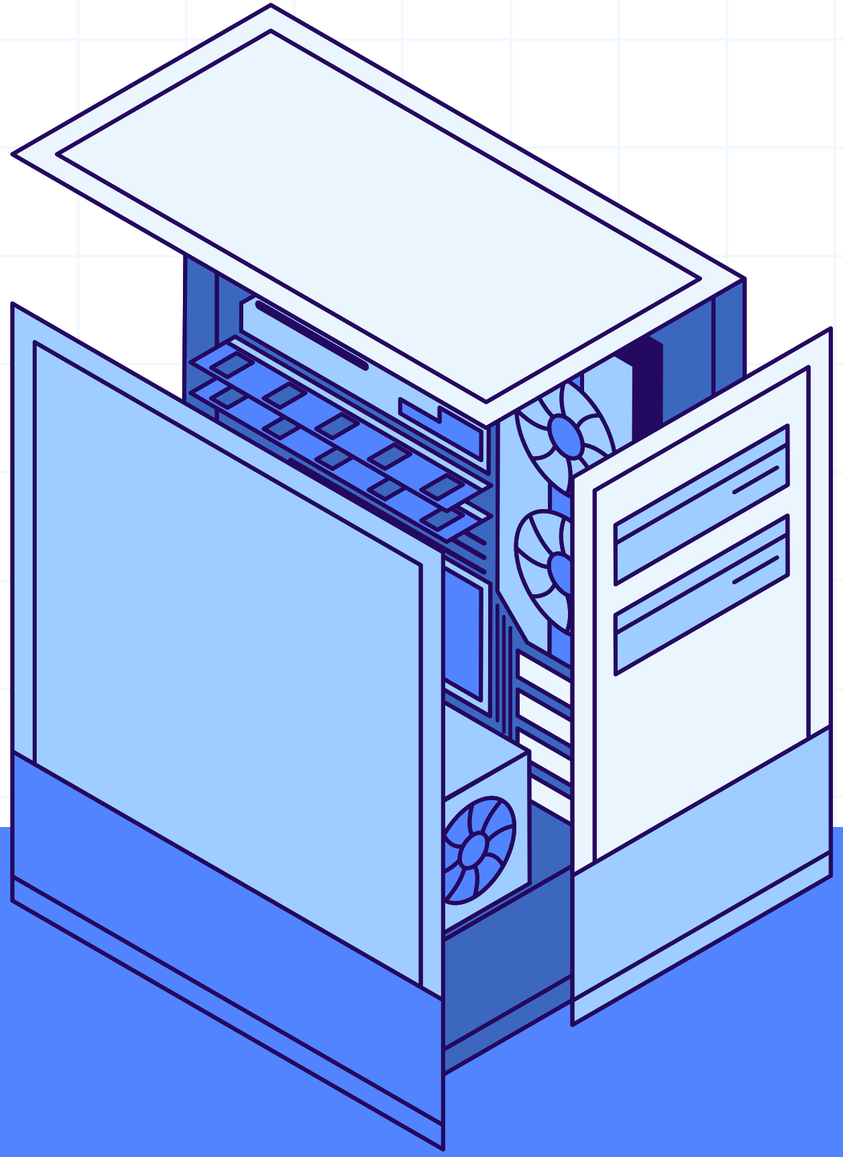
Condições da ULA (Zero, Negativo, Overflow, Carry), controla fluxo.

INTERFACE GRÁFICA: DADOS AO RASTERIZADOR



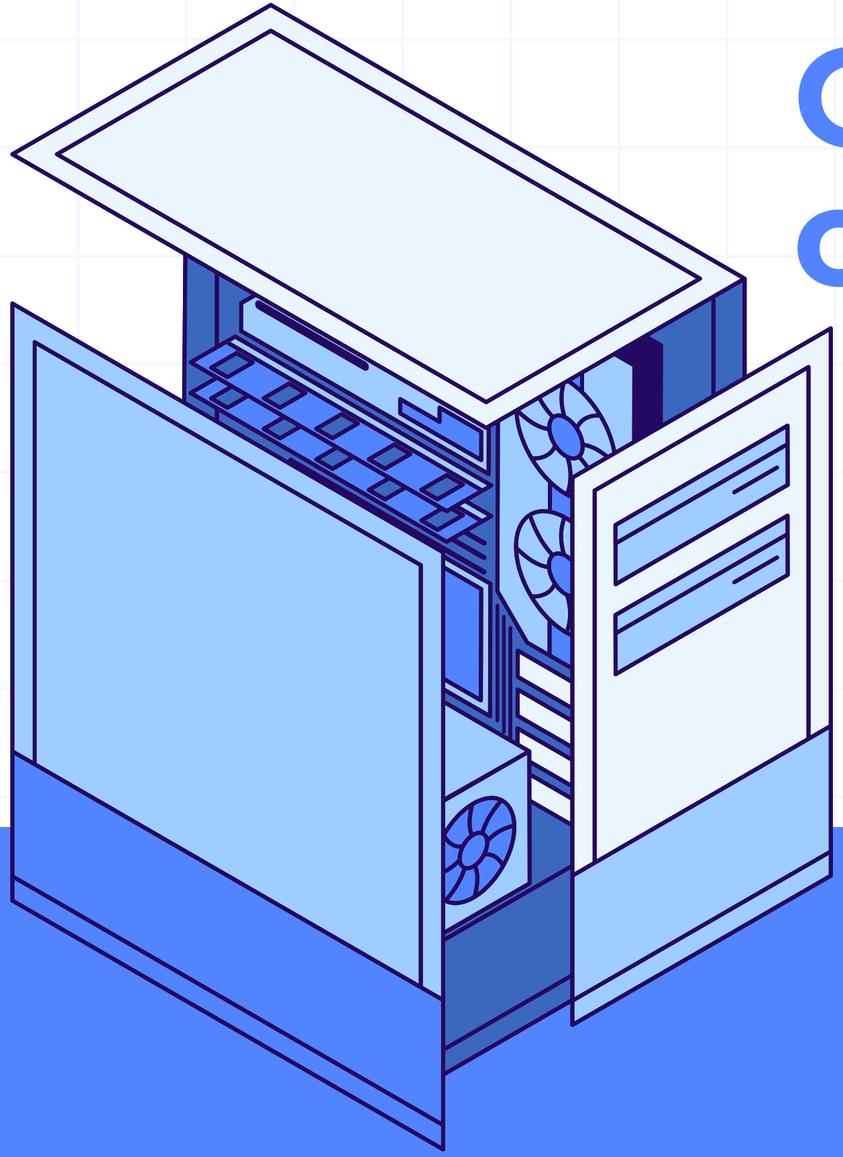
Alimenta motor gráfico com coordenadas de vértices, cor e paleta.

Gemini Notebook



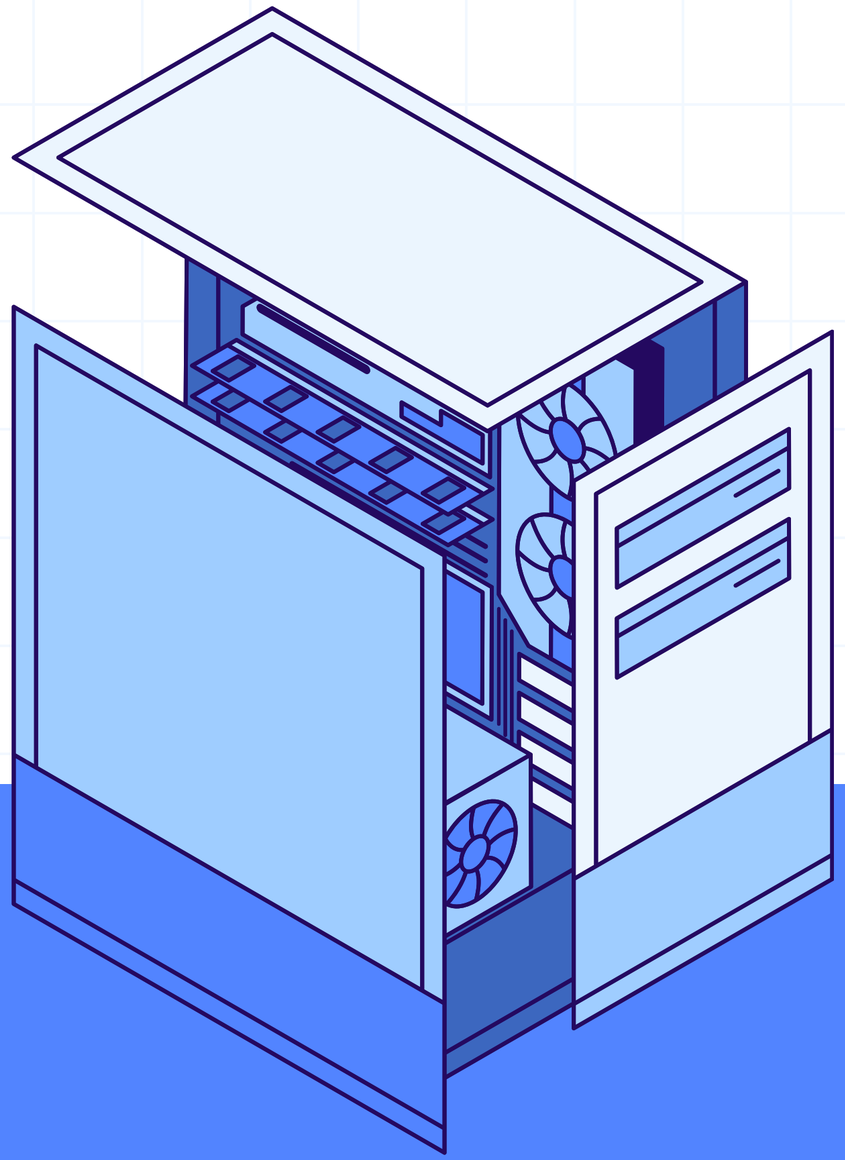
Unidade Lógica e Aritmética (ULA)

- A ULA (u_alu) recebe o primeiro operando por reg_a e o segundo por alu_b.
- O sinal alu_op define a operação que será executada.
- alu_shamt fornece a quantidade de deslocamento para operações que a utilizam.
- O resultado é disponibilizado em alu_rd.
- Os sinais alu_z, alu_n, alu_v e alu_c representam indicadores de estado: zero, negativo, overflow e carry.
- Ao concluir a operação, alu_done permite que o banco de registradores grave o resultado e atualize as flags.



Caminho de dados para configuração gráfica

- Instruções gráficas podem transportar dados do banco de registradores diretamente para as memórias e os motores.
- A unidade de controle gera sinais específicos de escrita e define os endereços e valores.
- As memórias gráficas são configuradas antes ou durante a execução das operações de desenho.



Motores gráficos e geração dos pixels

Motor de background (u_motor_bg)
Lê o tilemap, consulta os padrões de tiles e a paleta, considera o deslocamento do cenário e gera pixels com coordenadas e cores.

Motor de sprites (u_motor_sprite)
Utiliza posições, atributos e padrões dos sprites para gerar pixels. Considera seleção de paleta, transparência e inversões horizontal e vertical.

Rasterizador (u_rasterizador)
Recebe os parâmetros geométricos do banco de registradores e gera os pixels das primitivas, como retângulos e triângulos.

Bloco II: Motores de Renderização

motor_background

Entrada:
Coordenadas lógicas bidimensionais + deslocamento de scroll.

Função:
Varre sequencialmente a tela, lê o tilemap e converte em cor via consulta à paleta.

motor_sprite

Entrada:
spos_wr_data (17 bits) + sattr_wr_data (15 bits).

Função:
Processa até 32 sprites independentes, gerenciando prioridade relativa, relativa, espelhamento físico e transparência nativa (índice 0).

rasterizador_top

Entrada:
Vértices absolutos (v0, v1, v2) injetados do banco de registradores.

Função:
Atua como controlador dinâmico despachando trabalho matemático para rasterizador_quadrado ou rasterizador_triangulo.

Saída Unificada: fb_we + fb_wr_x/y + fb_wr_data

Gemini Notebook

Multiplexação dos dados gráficos

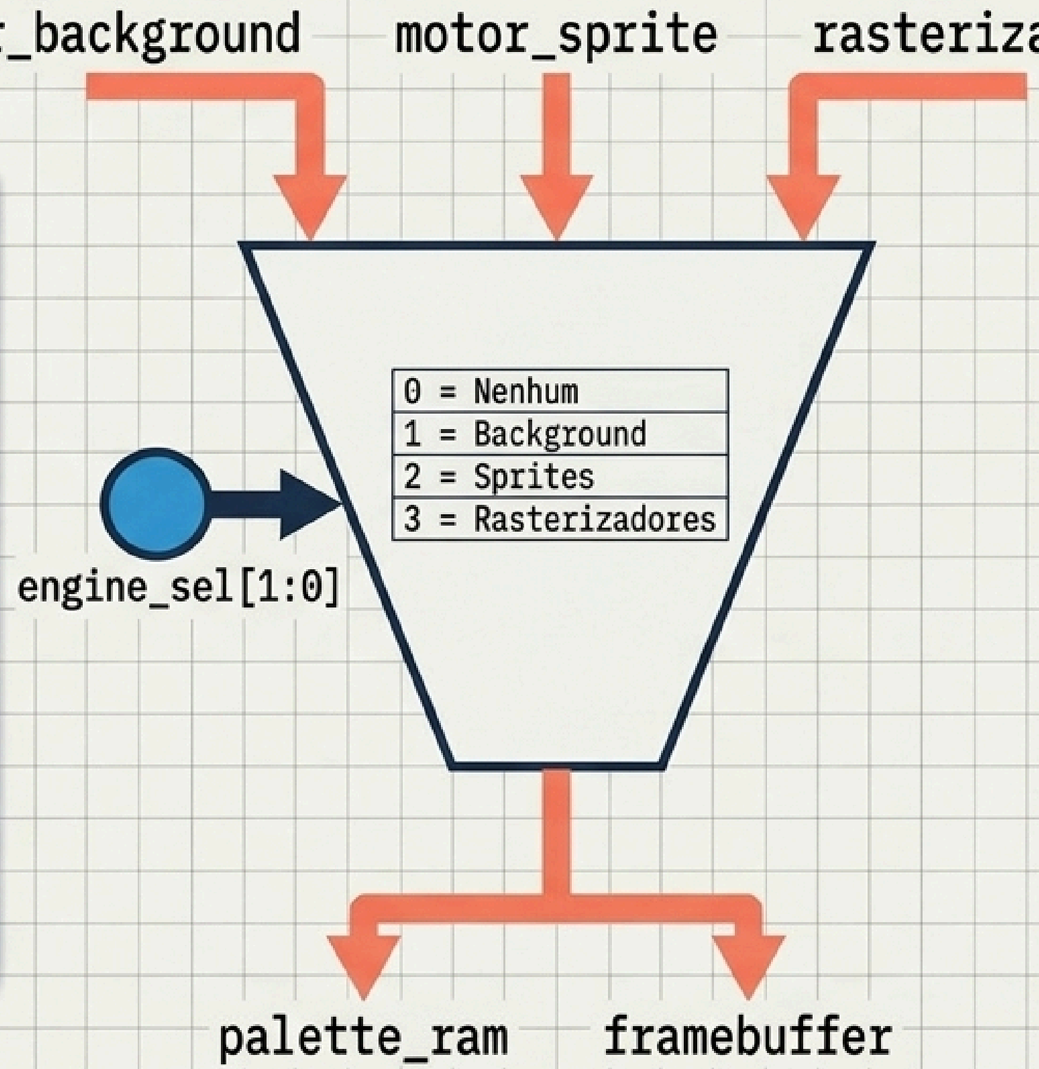
- Cada motor gera sinais relacionados à leitura da paleta e à escrita de pixels.
- O módulo engine_mux recebe os sinais dos motores e o seletor engine_sel.
- O multiplexador escolhe o endereço de leitura da paleta (pal_rd_addr) e os sinais de escrita de pixels (fb_we, fb_wr_x, fb_wr_y, fb_wr_data) do motor selecionado.
- A saída da paleta (pal_rd_data) é compartilhada com os motores.
- O multiplexador evita que os motores encaminhem simultaneamente suas saídas como se fossem um único fluxo de dados.

Bloco III: O Funil de Dados (engine_mux)

A Necessidade do Gargalo

O sistema compartilha a porta única de leitura da memória de paleta e a interface isolada de escrita do framebuffer.

O hardware garante que somente um motor possua direito de uso por ciclo.



Difusão Reversa

O engine_mux concentra apenas as solicitações de endereço (pal_rd_addr[8:0]). Uma vez processado, o dado resolvido (palette_rd_data) é transmitido de volta via barramento comum a todos os motores simultaneamente.

Gemini Notebook

Bloco III: Camada de Composição e Overlays

O Problema Temporal

A ordem linear das instruções Assembly (ex: executar DRAW_BG antes de DRAW_SPRITES) causaria sobreposição incorreta de elementos no buffer físico se escrita cegamente.

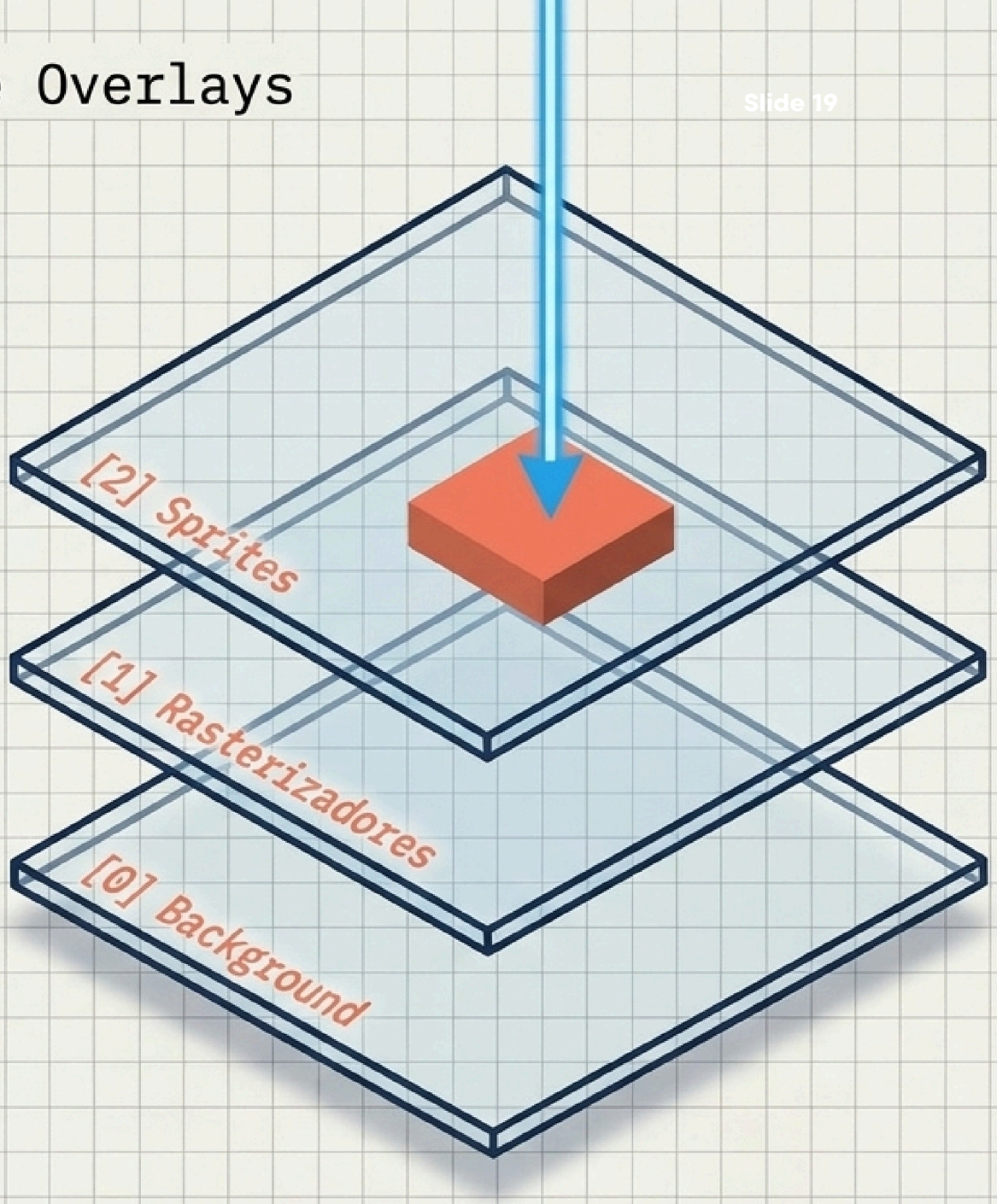
A Regra Matemática (Z-Buffer)

A lógica de hardware verifica os mapas antes de autorizar a escrita. A equação fundamental que governa a gravação do pixel é:

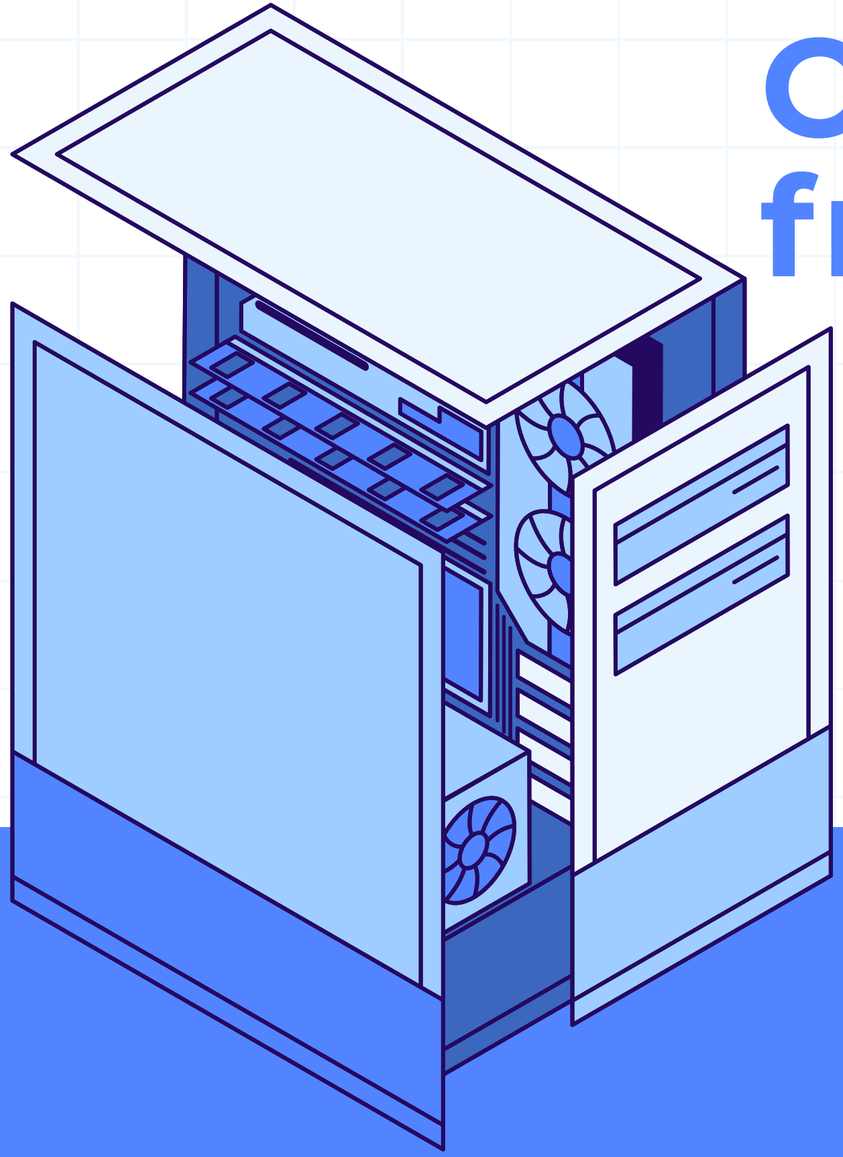
`camada nova >= camada registrada`

O Controle de Estado

Estes metadados espaciais são mantidos pela **mapa_camadas_ram** operando em paralelo aos buffers, guardando exatamente 2 bits (indicando camada) para os 76.800 pixels estruturais.

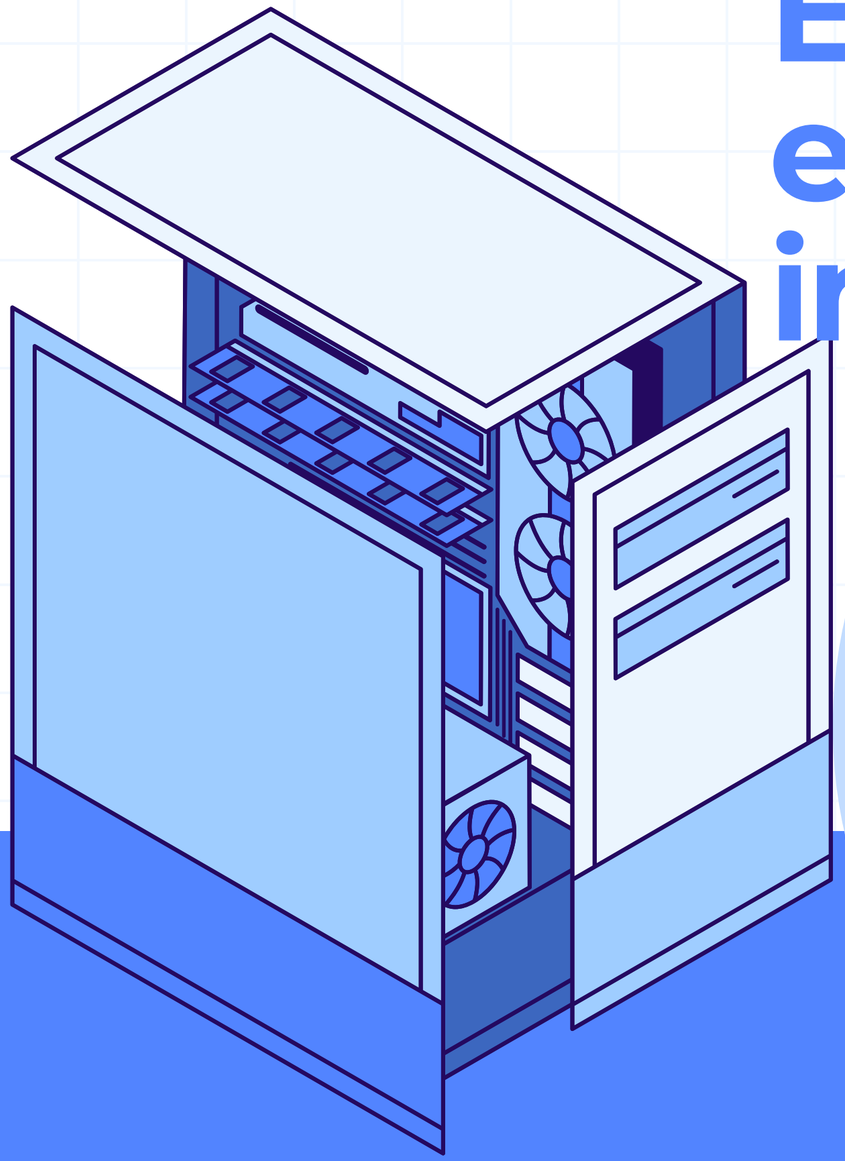


Gemini Notebook



Composição, framebuffer e saída VGA

- A camada de composição (u_composicao) recebe os pixels encaminhados pelo multiplexador.
- Ela organiza a composição visual e resolve a prioridade entre background, primitivas rasterizadas e sprites.
- Os sinais finais de escrita (fb_we, fb_wr_x, fb_wr_y, fb_wr_data) são enviados ao framebuffer.
- O framebuffer armazena a imagem em memória de pixels, usando cores de 9 bits no formato RGB reduzido.
- O driver VGA lê os dados armazenados e gera os sinais de sincronização e as saídas de cor para o monitor.

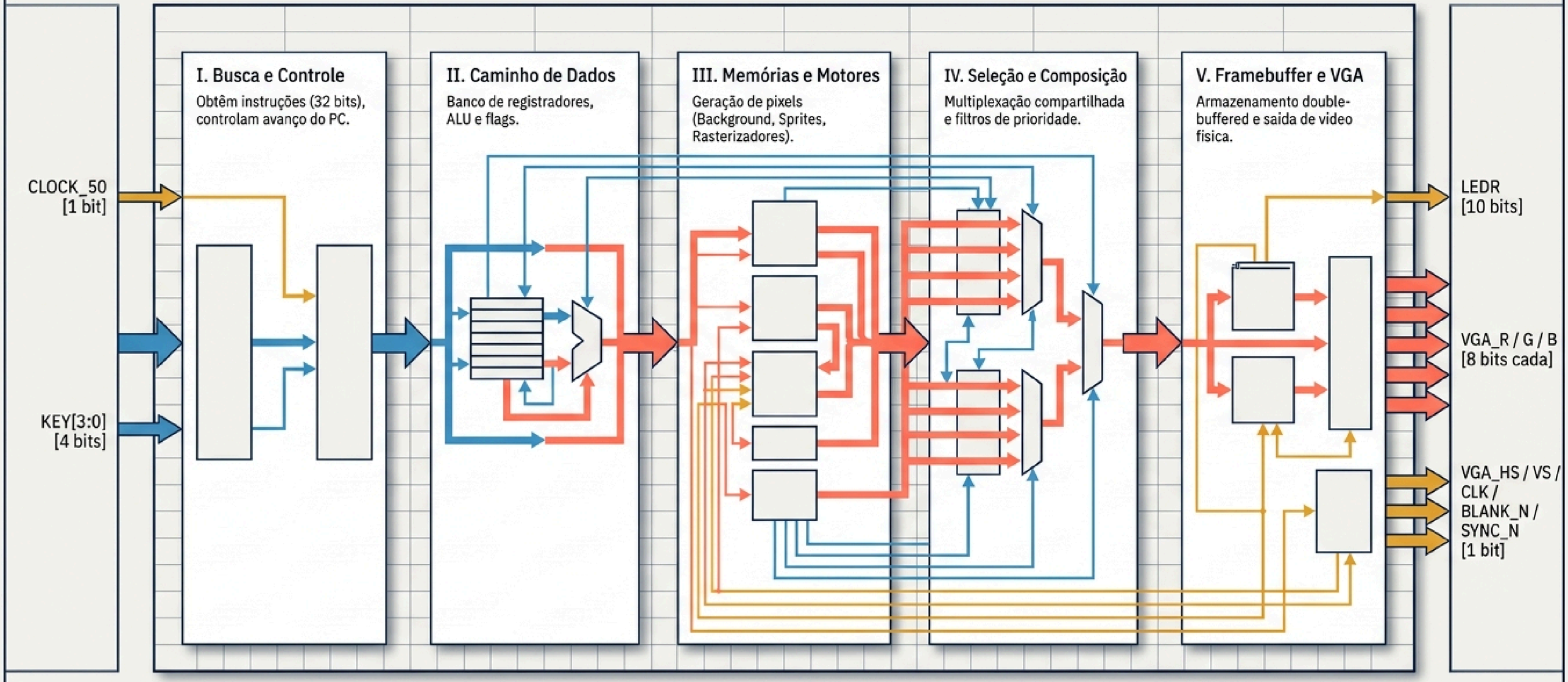


Exemplo completo: execução de uma instrução de desenho

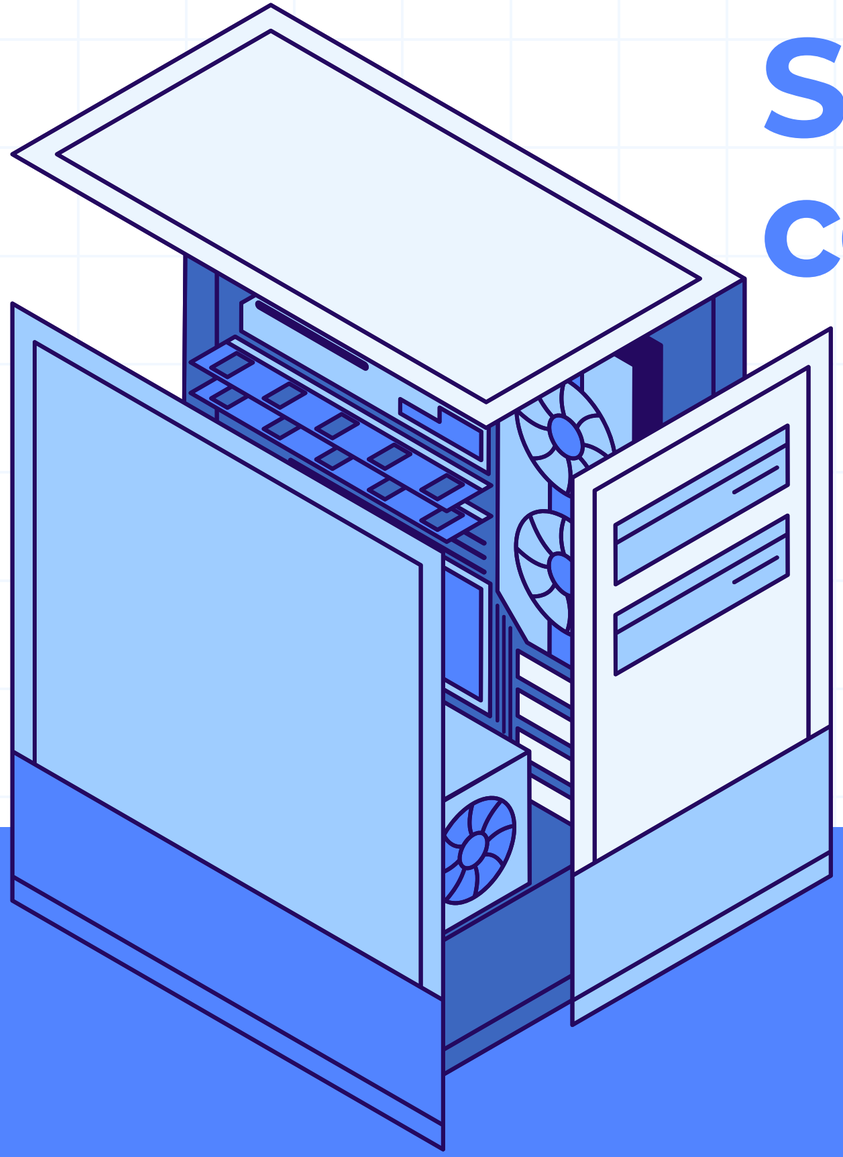
Considere uma instrução DRAW_RECT, utilizada para iniciar o desenho de um retângulo.

- Busca: a unidade de busca fornece a instrução.
- Decodificação: a unidade de controle identifica o opcode de DRAW_RECT.
- Leitura dos parâmetros: o banco de registradores disponibiliza os parâmetros geométricos usados pelo rasterizador.
- Inicialização: start_square ativa o rasterizador para iniciar a operação.
- Geração dos pixels: o rasterizador calcula quais pixels pertencem à primitiva e consulta a paleta para obter suas cores.
- Encaminhamento: o engine_mux seleciona os sinais de saída do rasterizador.
- Composição e escrita: a camada de composição encaminha os pixels ao framebuffer, respeitando as regras de composição.
- Conclusão: done_raster ou raster_invalid contribui para a geração de engine_done, permitindo à unidade de busca prosseguir conforme a lógica de controle.

Arquitetura Macro: coprocessador_top.v



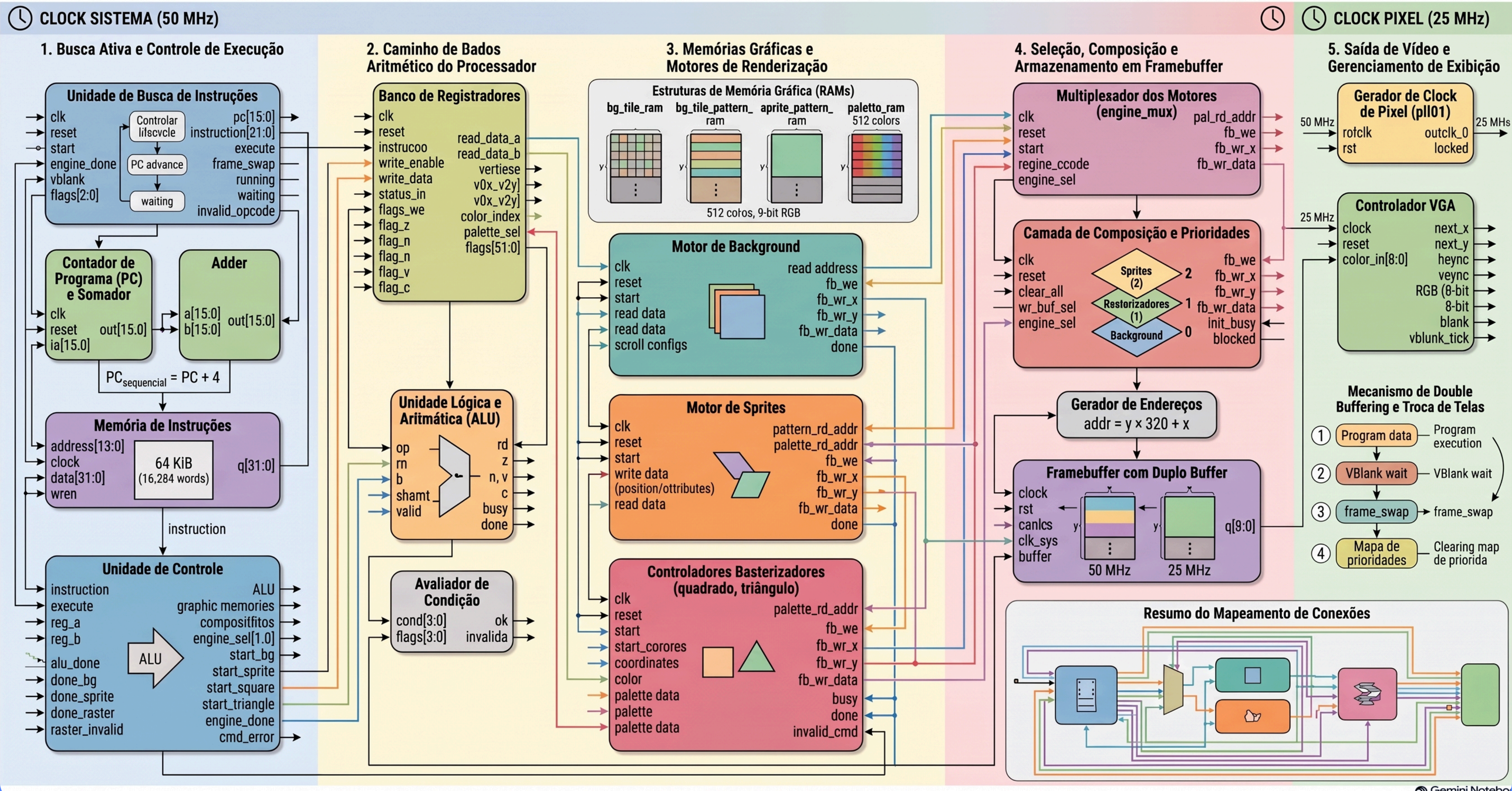
Gemini Notebook

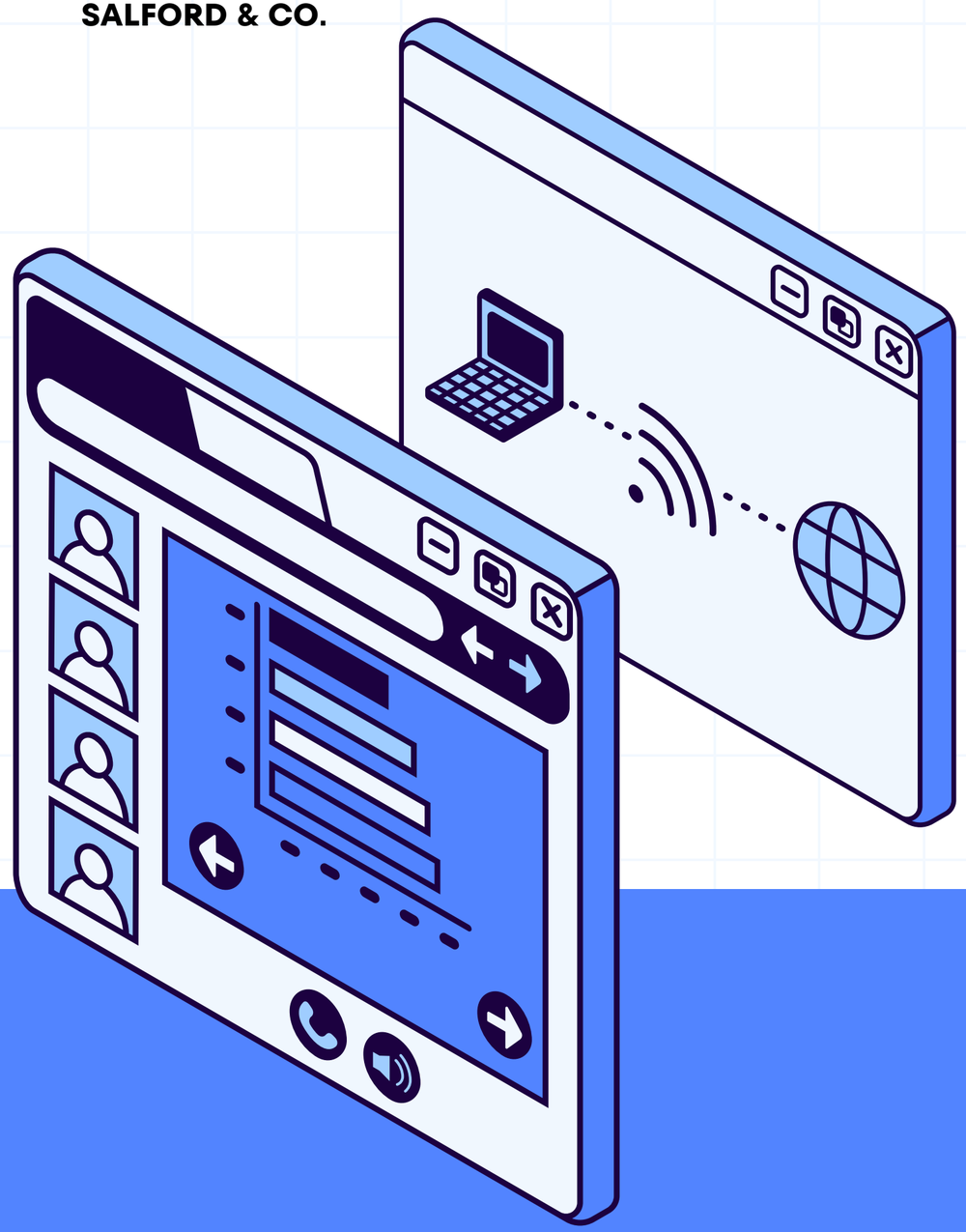


Síntese: separação entre controle e datapath

- Datapath: transporta e processa operandos, parâmetros, cores, coordenadas e pixels.
- Unidade de controle: decodifica as instruções, habilita as operações e coordena os motores.
- Banco de registradores: fornece operandos e parâmetros e armazena resultados da ULA.
- Memórias gráficas: fornecem os dados necessários para a geração de imagens.
- Motores gráficos: convertem parâmetros e dados gráficos em pixels.
- Caminho de saída: seleciona, compõe e armazena os pixels para exibição via VGA.

Arquitetura e Datapath do Coprocessador Gráfico em FPGA





Instruction Set Architeture

- ISA de 32 bits: instruções de tamanho fixo, com opcode de 5 bits.
- Busca ativa: o coprocessador busca e executa instruções armazenadas em sua memória interna, carregadas por arquivo .mif.
- Conjunto de instruções: operações aritméticas e lógicas, configuração gráfica, desenhos geométricos e controle de sprites.
- Objetivo: permitir que um programa controle os recursos gráficos sem precisar gerenciar diretamente cada detalhe do hardware.

ISA
FORMATOS e operações

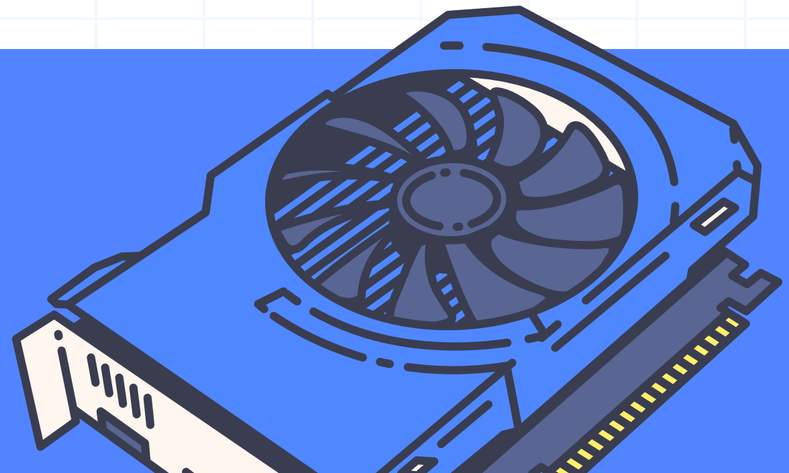
OPCODES

ISA do Coprocessador Gráfico (DE1-SoC / Verilog)
Tabela de Opcodes

Opcode (5 bits)	Instrução	Formato
00000	NOP	N
00001	ADD	A
00010	SUB	A
00011	AND	A
00100	LSL	A
00101	LSR	A
00110	PASS	A
00111	SET_PALETTE	G
01000	DRAW_RECT	N
01001	DRAW_TRI	N
01010	SET_SPRITE_POS	G
01011	SET_SPRITE_ATTR	G
01100	SET_TILEMAP	G
01101	SCROLL_BG	G
01110	WRITE_SPRITE_DATA	G
01111	WRITE_TILE_DATA	G
10000	DRAW_BG	N
10001	DRAW_SPRITES	N
10010	WAIT_VBLANK	N
10011	HALT	N
10100	B	D
10101	B_COND	D
10110 – 11111	Reservados (LOAD/STORE, BL/RET, etc.)	–

Principais operações gráficas:

- A — ALU: operações aritméticas e lógicas, como ADD, SUB e PASS.
- G — Gráficas: configuração de recursos gráficos com operandos em registradores.
- N — Sem operandos: comandos como DRAW_TRI, DRAW_RECT e HALT.
- D — Desvios: controle do fluxo de execução com B e B_COND.



ISA

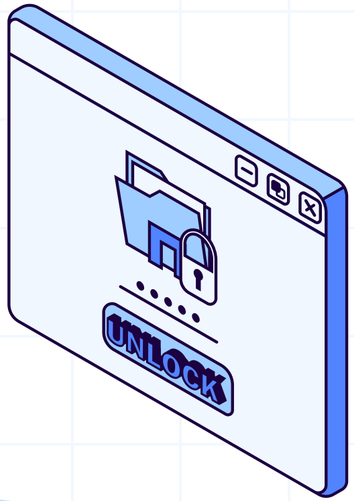
Execução e controle gráfico

Busca ⟶ **Decodificação** ⟶ **Execução** ⟶ **Controle**

Sincronização gráfica:

- As instruções de desenho aguardam a conclusão do motor gráfico (DONE).
- WAIT_VBLANK sincroniza a execução com o período de apagamento vertical.
- HALT encerra o programa do quadro; no próximo VBLANK, ocorre a troca dos framebuffers e a execução reinicia no endereço zero.

Assembly Movimentação dos Sprites



Inicialização

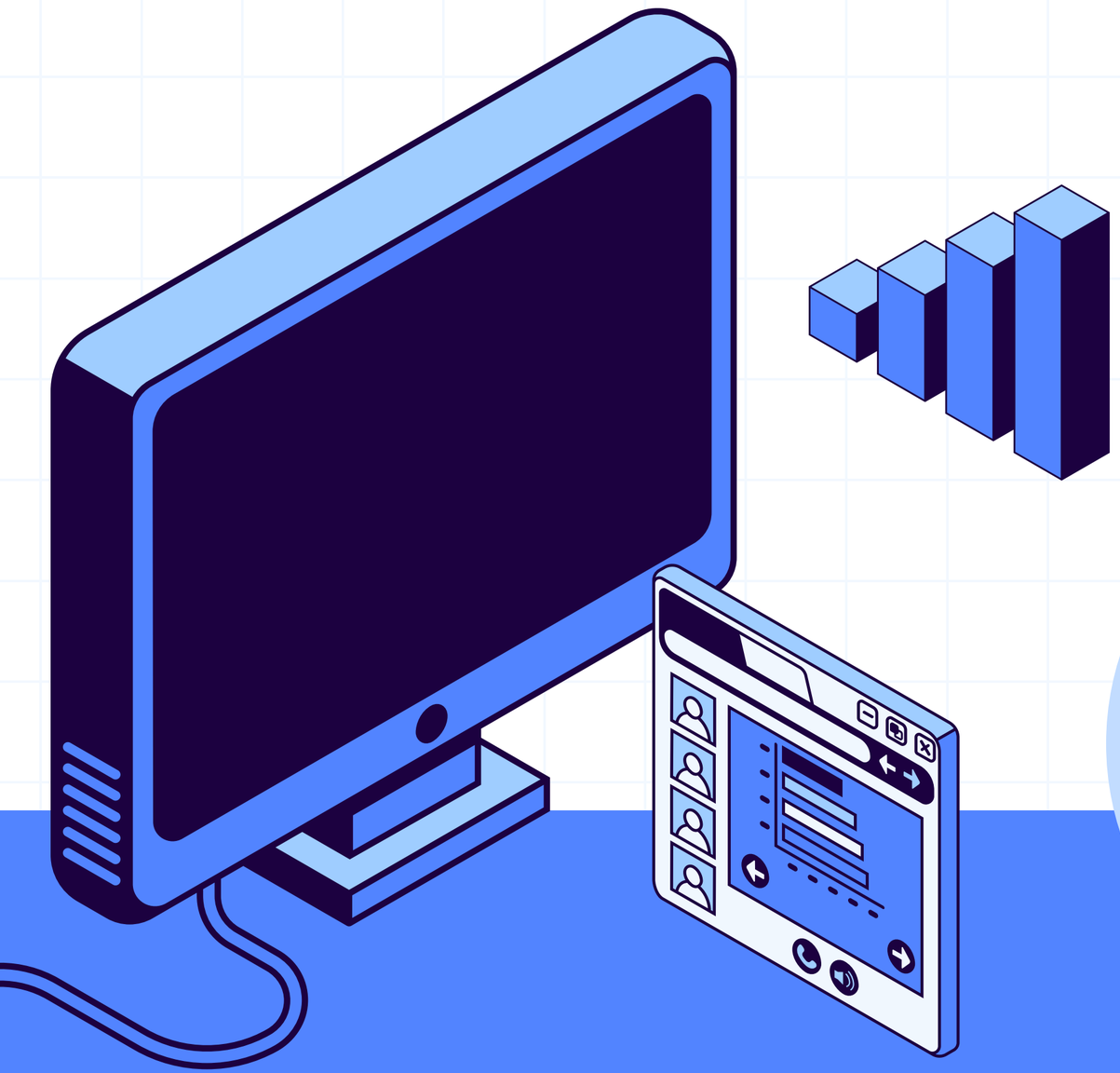
- Define posição (x, y) e velocidades (dx, dy).
- Preserva o estado entre quadros.

Movimento e Colisões

- ADD: atualiza as coordenadas.
- SUB e B_COND: detectam colisões com as bordas.
- Ao atingir uma borda, inverte a velocidade e alterna o espelhamento.

Instruções principais

- SET_SPRITE_POS: atualiza a posição.
- SET_SPRITE_ATTR: configura os atributos do sprite.



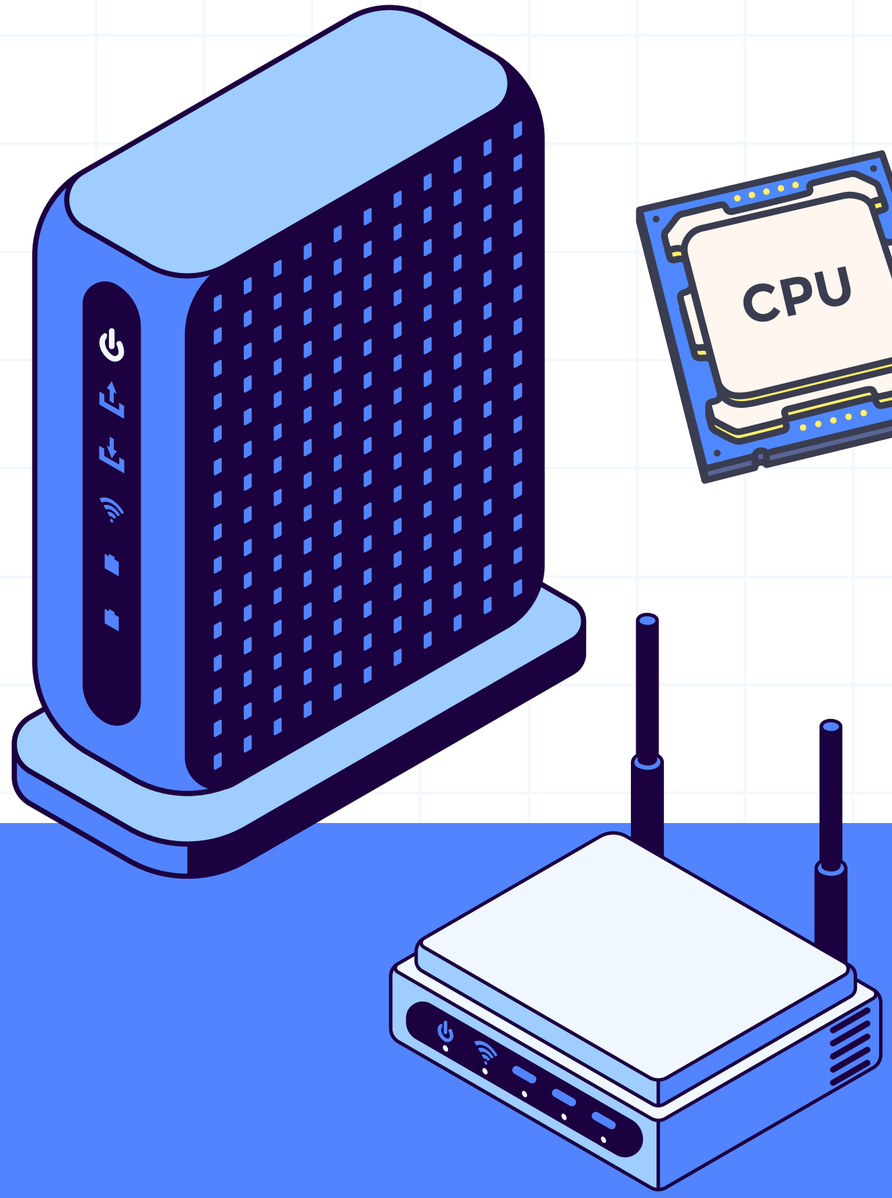
Assembly
Sprites e formas geométricas

- **4 sprites:** Sonic e Shadow, dois animados e dois estáticos.
- **SET_SPRITE_POS e SET_SPRITE_ATTR:** configuram posição e aparência dos sprites.
- **DRAW_RECT e DRAW_TRI:** desenham um quadrado e um triângulo usando os registradores dedicados.

Assembly
Composição do frame

- **DRAW_BG:** desenha o plano de fundo.
- **DRAW_RECT e DRAW_TRI:** desenham as formas geométricas.
- **DRAW_SPRITES:** renderiza os sprites.
- **HALT:** encerra a execução do programa no quadro atual.

- **Prioridade visual:** plano de fundo → formas geométricas → sprites.



Assembly SIMULAÇÃO

Fizemos um simulador do funcionamento do coprocessador para códigos Assembly em html

Obrigado pela atenção!

REFERÊNCIAS:

- LEGv8 VISUAL SIMULATOR. LEGv8 Visual Simulator. [S. l.], [s. d.]. Disponível em: https://hcmus-hqhuy.github.io/LEGv8-Simulator/. Acesso em: 14 out. 2026.
- UNIVERSIDADE ESTADUAL DE FEIRA DE SANTANA. Departamento de Tecnologia. Área de Eletrônica. Problema #2 – 2026.2: Sistema Digital. Feira de Santana: UEFS, 2026.